

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: Database Management Systems
COURSE CODE: CSA05

COURSE OUTCOME COVERED

CO5: *Implement database operations using query processing, optimization, and transaction management to ensure consistency and reliability.* (BL3, BL4)

Activity Tool : Constraint-Based Problem Solving (High)
Assessment Tool Weightage: 35%

Faculty Name: Dr.G.Ayyappan

Student Reg.No: 192424061
Name:K.Sandhya

QUESTIONS			Total Marks: 50
Q.No	Question	Bloom's Level	Marks
1	Given a complex SQL query with nested joins and sub-queries, draw the query execution plan and apply heuristic optimization rules step-by-step.	BL4 – Analyze	5
2	Apply cost-based optimization to choose between nested-loop join, sort-merge join, and hash join for the given relations with specified tuple counts and page sizes.	BL4 – Analyze	5
3	Implement JDBC connectivity in Java to a MySQL database and write code to execute parameterized queries with PreparedStatement.	BL3 – Apply	5
4	Given a schedule of four concurrent transactions, determine whether it is conflict-serializable using a precedence graph.	BL4 – Analyze	5
5	Demonstrate the application of Basic Two-Phase Locking (2PL) on a given transaction set. Identify and resolve any deadlocks.	BL3 – Apply	5
6	Apply the Wait-Die and Wound-Wait timestamp-based deadlock prevention schemes on a given set of transactions requesting locks.	BL4 – Analyze	5
7	Given transaction logs with checkpoints, apply the Immediate Update recovery protocol to recover the database after a system failure.	BL3 – Apply	5

8	Apply Deferred Update (NO-UNDO/REDO) recovery technique on a given log file with committed and uncommitted transactions.	BL3 – Apply	5
9	Design a concurrency control mechanism using strict 2PL for a banking system where T1 transfers funds and T2 reads balance simultaneously.	BL4 – Analyze	5
10	Write pseudocode for query optimization using heuristics: push down selection, project early, and order joins by smaller intermediate results.	BL4 – Analyze	5

Code of Conduct:

I _K.Sandhya 192424061_ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

RUBRICS FOR CALCULATION:

Constraint-Based Problem Solving Assessment Rubric					
Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	25%	Clearly defines the problem and identifies all relevant constraints accurately	Identifies most constraints with minor gaps	Partial understanding; misses key constraints	Fails to identify constraints correctly
Constraint Analysis	25%	Analyzes constraints effectively and prioritizes them logically	Provides reasonable analysis with some prioritization	Limited analysis; weak prioritization	No meaningful analysis or prioritization
Solution Development	25%	Develops optimal and feasible solutions satisfying all constraints	Develops workable solutions with minor limitations	Solutions partially satisfy constraints	Solutions do not meet constraints
Application of Concepts	15%	Effectively applies appropriate concepts, methods, or tools	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply relevant concepts

Justification	10%	Provides strong justification for decisions with logical reasoning	Provides justification with some clarity	Weak or unclear justification	No justification provided
---------------	-----	--	--	-------------------------------	---------------------------

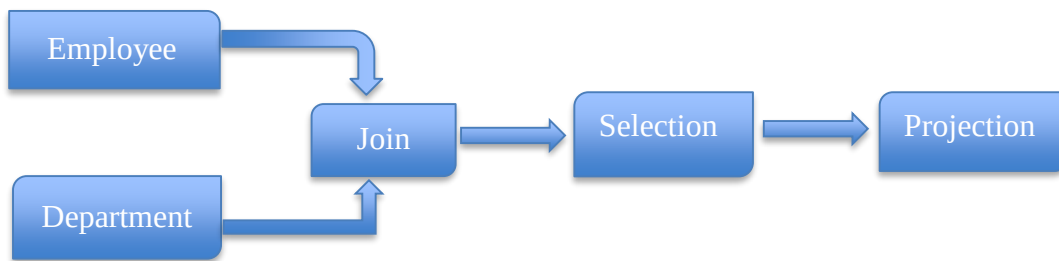
1. Query Execution Plan and Heuristic Optimization

A query execution plan is the sequence of operations used by a DBMS to execute an SQL query.

Example Query

```
SELECT E.Name, D.DeptName
FROM Employee E
JOIN Department D ON E.DeptID = D.DeptID
WHERE E.Salary > 50000
AND D.Location = 'Chennai';
```

Initial execution plan



Heuristic optimization steps

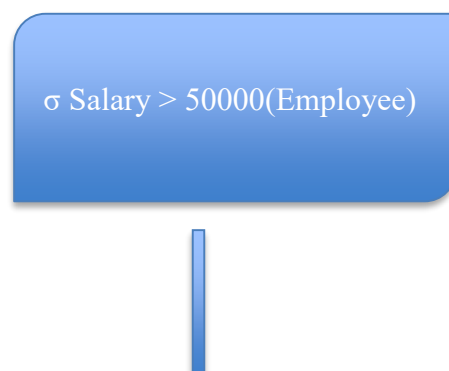
Step 1 – Push selections down

Apply conditions as early as possible.

$\sigma_{\text{Salary} > 50000}(\text{Employee})$

$\sigma_{\text{Location} = \text{'Chennai'}}(\text{Department})$

Step 2 – Perform smaller join





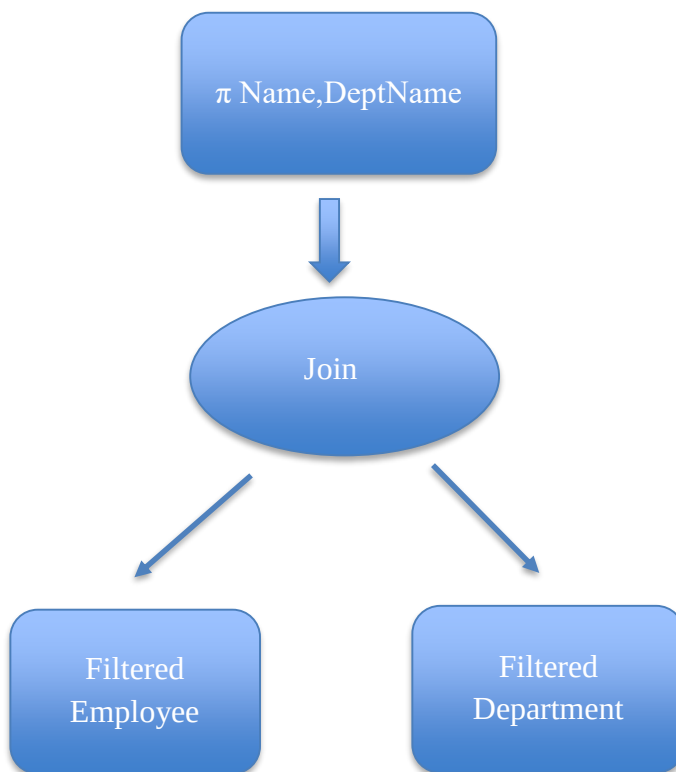
Step 3 – Push projections down

Keep only required attributes:

$\pi(\text{Name}, \text{DeptID})(\text{Employee})$

$\pi(\text{DeptID}, \text{DeptName})(\text{Department})$

Step 4 – Join the reduced relations



```
mysql> CREATE DATABASE college;
Query OK, 1 row affected (0.00 sec)
```

```
mysql> USE college;
Database changed
```

```
mysql> CREATE TABLE Student (
->     ID INT PRIMARY KEY,
->     Name VARCHAR(50),
->     DeptID INT,
->     Mark INT
-> );
```

```
mysql> CREATE TABLE Department (
->     DeptID INT PRIMARY KEY,
->     DeptName VARCHAR(50)
-> );
Query OK, 0 rows affected (0.00 sec)
```

```
mysql> INSERT INTO Student VALUES
-> (1, 'Ravi', 10, 80),
-> (2, 'Priya', 20, 40),
-> (3, 'Arun', 10, 90);
Query OK, 3 rows affected (0.02 sec)
Records: 3  Duplicates: 0  Warnings: 0
```

```
mysql>
mysql> INSERT INTO Department VALUES
-> (10, 'CSE'),
-> (20, 'ECE');
Query OK, 2 rows affected (0.00 sec)
Records: 2  Duplicates: 0  Warnings: 0
```

```
mysql> SELECT * FROM Student;
+----+-----+-----+-----+
| ID | Name  | DeptID | Mark |
+----+-----+-----+-----+
| 1  | Ravi  | 10     | 80   |
| 2  | Priya | 20     | 40   |
| 3  | Arun  | 10     | 90   |
+----+-----+-----+-----+
3 rows in set (0.00 sec)
```

```
mysql> SELECT * FROM Department;
+-----+-----+
| DeptID | DeptName |
+-----+-----+
|      10 | CSE      |
|      20 | ECE      |
+-----+-----+
2 rows in set (0.00 sec)
```

```
mysql> SELECT S.Name, D.DeptName
-> FROM Student S
-> JOIN Department D
-> ON S.DeptID = D.DeptID
-> WHERE S.Mark > 50;
+-----+-----+
| Name | DeptName |
+-----+-----+
| Ravi | CSE      |
| Arun | CSE      |
+-----+-----+
2 rows in set (0.00 sec)
```

```
SELECT S.Name, D.DeptName
FROM
  (SELECT Name, DeptID
   ' at line 1
mysql> SELECT S.Name, D.DeptName
-> FROM
->      (SELECT Name, DeptID
->        FROM Student
->        WHERE Mark > 50) S
-> JOIN Department D
-> ON S.DeptID = D.DeptID;
+-----+-----+
| Name | DeptName |
+-----+-----+
| Ravi | CSE      |
| Arun | CSE      |
+-----+-----+
2 rows in set (0.00 sec)
```

Result

The optimized query processes fewer tuples, requires less memory, and reduces disk I/O.

2. Cost-Based Optimization of Joins

Suppose:

Relation	Tuples	Pages
R	1,000	100
S	10,000	1,000

Assume 10 buffers.

1. Nested-Loop Join

Approximate block nested-loop cost:

$$\begin{aligned} & B(R) + [B(R)M - 2]B(S) \\ &= 100 + [100/8](1000) = 100 + \lceil 100/8 \rceil (1000) \\ &= 100 + 13(1000) = 13,100 \end{aligned}$$

2. Sort-Merge Join

Approximate cost includes sorting both relations and merging them.

It generally requires several passes over the relations, so its cost is higher when relations are not already sorted.

3. Hash Join

Hash join partitions both relations and then matches corresponding partitions.

Approximate cost:

$$3(B(R) + B(S)) = 3(100 + 1000) = 3300$$

Comparison

Join	Approximate Cost
Nested Loop	13,100

Join	Approximate Cost
Sort-Merge	Higher than hash in this example
Hash Join	3,300

Conclusion

Hash Join is selected because it has the lowest estimated I/O cost for equality joins when sufficient memory is available.

3. JDBC Connectivity with MySQL

JDBC (Java Database Connectivity) allows Java programs to communicate with databases such as MySQL.

Steps

1. Import JDBC classes.
2. Load/connect to MySQL.
3. Create a Connection.
4. Use PreparedStatement.
5. Execute the query.
6. Process the result.
7. Close the connection.

Code:

```
import java.sql.*;

public class JDBCExample {

    public static void main(String[] args) {

        String url = "jdbc:mysql://localhost:3306/college";

        String user = "root";

        String password = "root";
```

```
String sql = "SELECT * FROM Student WHERE Department = ?";
```

```
try {
```

```
    Connection con = DriverManager.getConnection(
```

```
        url, user, password);
```

```
    PreparedStatement ps = con.prepareStatement(sql);
```

```
    ps.setString(1, "CSE");
```

```
    ResultSet rs = ps.executeQuery();
```

```
    while (rs.next()) {
```

```
        System.out.println(
```

```
            rs.getInt("Student_ID") + " " +
```

```
            rs.getString("Name"));
```

```
    }
```

```
    rs.close();
```

```
    ps.close();
```

```
    con.close();
```

```
    } catch (SQLException e) {
```

```
        System.out.println(e);
```

```
    }
```

```
}
```

```
}
```

Advantage of Prepared Statement

- Prevents SQL injection.
- Supports parameterized queries.
- Improves security.
- Can improve performance for repeated queries.

4. Conflict Serializability Using Precedence Graph

A schedule is conflict-serializable if its precedence graph contains no cycle.

Suppose the schedule is:

T1: R(A) W(A) R(B)

T2: R(A) W(A)

T3: R(B) W(B)

T4: R(A)

Steps

Step 1: Create one node for every transaction.

T1 T2 T3 T4

Step 2: Identify conflicting operations.

Conflicts occur when:

- They belong to different transactions.
- They access the same data item.
- At least one operation is a write.

Step 3: Add directed edges.

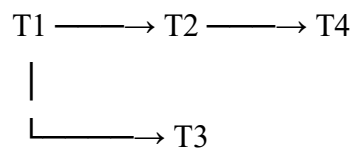
For example:

T1 → T2

T1 → T3

$T2 \rightarrow T4$

Precedence graph



Conclusion

There is no cycle, so the schedule is conflict-serializable.\

5. Basic Two-Phase Locking (2PL)

Two-Phase Locking ensures conflict serializability by dividing a transaction into two phases.

Two phases

1. Growing phase

- Transaction can acquire locks.
- Transaction cannot release locks.

2. Shrinking phase

- Transaction can release locks.
- Transaction cannot acquire new locks.

Example

T1: LOCK-X(A)

READ(A)

WRITE(A)

LOCK-X(B)

READ(B)

WRITE(B)

UNLOCK(A)

UNLOCK(B)

Here:

Growing Phase



LOCK A → LOCK B



Shrinking Phase



UNLOCK A → UNLOCK B

Deadlock example

T1: LOCK-X(A)

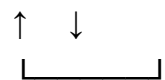
T2: LOCK-X(B)

T1: requests B → waits

T2: requests A → waits

This creates:

T1 → T2



Resolution

The DBMS can abort one transaction, for example **T2**.

Abort T2



Release B



T1 obtains B



T1 completes

6. Wait-Die and Wound-Wait

Both are timestamp-based deadlock prevention techniques.

Assume:

T1 timestamp = 10 → Older

T2 timestamp = 20 → Younger

Suppose T2 holds a lock and T1 requests it.

Wait-Die

Older transaction waits; younger transaction dies.

T1 is older and requests a lock held by T2:

T1 → waits for T2

If T2 requests a lock held by T1:

T2 → aborts

So:

Older → Wait

Younger → Die

Wound-Wait

Older transaction forces younger transaction to abort; younger waits.

If T1 requests a lock held by T2:

T1 is older

↓
T2 is aborted
↓
T1 gets the lock

If T2 requests a lock held by T1:

T2 is younger
↓
T2 waits

Difference

Scheme	Older Requests Younger	Younger Requests Older
Wait-Die	Wait	Abort
Wound-Wait	Abort younger	Wait

Both schemes prevent circular waiting and therefore prevent deadlocks.

7. Immediate Update Recovery

In Immediate Update, database changes can be written before the transaction commits.

Therefore, recovery may require:

- UNDO for uncommitted transactions.
- REDO for committed transactions.

Example log

<T1 START>
<T1, A, 100, 150>

<T2 START>
<T2, B, 200, 250>

<T1 COMMIT>

<CHECKPOINT>

<T2, C, 300, 350>

SYSTEM FAILURE

Recovery

At the time of failure:

T1 → COMMITTED

T2 → UNCOMMITTED

Therefore:

T1 → REDO

T2 → UNDO

Result

REDO T1 changes

UNDO T2 changes

The database is restored to a consistent state.

8. Deferred Update – NO-UNDO/REDO

In Deferred Update, database changes are not written to the database until the transaction commits.

Therefore:

- Committed transactions → **REDO**
- Uncommitted transactions → **Ignore**

Example

<T1 START>

<T1, A, 100, 150>

<T1 COMMIT>

<T2 START>

<T2, B, 200, 250>

SYSTEM FAILURE

Recovery

T1 committed:

REDO T1

T2 did not commit:

IGNORE T2

Recovery table

Transaction	Status	Action
T1	Committed	REDO
T2	Uncommitted	Ignore

Conclusion

Deferred update is called NO-UNDO/REDO because uncommitted changes never reach the database.

9. Strict 2PL for Banking System

Consider a bank transfer:

T1: Transfer ₹100 from Account A to Account B

T2: Read Account A balance

T1

LOCK-X(A)

LOCK-X(B)

READ(A)

$A = A - 100$

WRITE(A)

READ(B)

$B = B + 100$

WRITE(B)

COMMIT

UNLOCK(A)

UNLOCK(B)

T2

LOCK-S(A)

READ(A)

COMMIT

UNLOCK(A)

Concurrent execution

Initially:

T1 obtains X-lock(A)

T1 obtains X-lock(B)

When T2 tries:

$T2 \rightarrow S\text{-lock}(A)$

T2 must wait because T1 has an exclusive lock.

T1

↓

Update A and B

↓

COMMIT

↓

Release locks

↓

T2 obtains S-lock(A)

↓

Read balance

Why Strict 2PL?

Strict 2PL holds all exclusive locks until commit/abort.

It prevents:

- Dirty reads
- Cascading rollback
- Lost updates
- Inconsistent bank balances

10. Pseudocode for Heuristic Query Optimization

Heuristic optimization improves a query by applying rules that generally reduce intermediate results.

Main rules

1. Push selections down.
2. Perform projections early.
3. Order joins to reduce intermediate results.

Pseudocode

Algorithm Heuristic_Optimization(Query)

Input:

SQL Query

Output:

Optimized Query Execution Plan

Step 1:

Parse the SQL query.

Step 2:

Create the initial query tree.

Step 3:

Find all selection conditions.

Step 4:

Push selection operations as close
as possible to the base relations.

Step 5:

Find required attributes.

Step 6:

Push projection operations down
to remove unnecessary attributes.

Step 7:

Identify all join operations.

Step 8:

Estimate the size of intermediate results.

Step 9:

Perform joins that produce smaller intermediate relations first.

Step 10:

Remove unnecessary operations.

Step 11:

Return the optimized query tree.

End

Example

Original:

Join

/ \

Join R3

/ \

R1 R2

If R1 JOIN R2 produces a very large result, but R2 JOIN R3 produces a smaller result, reorder:

Join

/ \

R1 Join

/ \

R2 R3

Benefits

- Reduces intermediate relations.

- Reduces disk I/O.
- Reduces memory usage.
- Improves query execution time.
- Makes query processing more efficient.